**MASTER 2 INFORMATIQUE - AVR**

# TRAITEMENT ET ANALYSE D'IMAGES
## Partie 1 : Analyse des contours et des formes dans les images

## TP1 – COMPOSANTES CONNEXES - FILTRAGE - COMPARAISON DE CONTOURS – CODE DE FREEMAN

**Les codes des programmes réalisés doivent accompagner le compte rendu et être déposés dans un fichier compressé sur arche.**

### PARTIE I – EXTRACTION DES COMPOSANTES CONNEXES DANS UNE IMAGE AVEC OPENCV

Plusieurs exemples accompagnent la documentation d'opencv (https://opencv.org/). Tester l'exemple *connected_components.cpp* avec les images *objet-publicitaire.jpg, objets-scolaires.jpg et Automne.jpg*.
Quel est le rôle de la variable threshval ? Commenter pour cela l'instruction suivante

```
Mat bw = threshval < 128 ? (img < threshval) : (img > threshval);
```

Quelles modifications apporter au programme pour obtenir une couleur différente par objet dans *objet-publicitaire.jpg* et *objets-scolaires.jpg*.

Consulter la documentation pour indiquer quels sont les algorithmes utilisés pour l'extraction des composantes connexes. Donner leurs références bibliographiques.

Proposer un programme simple sans utiliser de trackbar pour labelliser les composantes connexes des images binaires *dessin.pgm, JuliaSet.pgm, spots.pgm, test.pgm*. Deux résultats sont possibles pour *test.pgm*, expliquer pourquoi.

### PARTIE II – QUELQUES FILTRES AVEC OPENCV

#### II.1. FILTRES DE CONVOLUTION - FILTRE MEDIAN – FILTRE BILATERAL

*Rappels issus du site de B. Perret[1], des compléments sont aussi disponibles dans le cours TIVO de Master 1 sur Arche accessible par tous.*

Formellement, le **produit de convolution** est une opération entre deux images en niveau de gris $f$ et $g$, notée $f*g$ défini par :

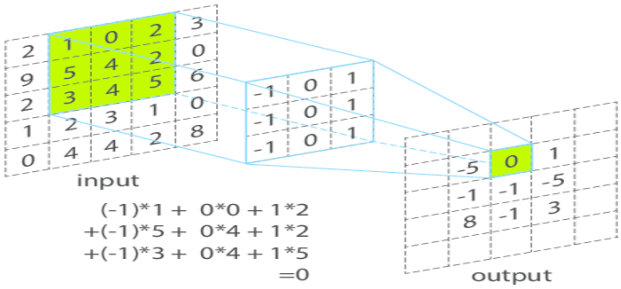
$$\forall (x,y)\in\mathbb{Z}2, (f*g)(x,y)=\sum_i \sum_j f(i,j)g(x-i,y-j)$$

Dans cette expression on peut reconnaître :

- $g(x-i,y-j)$ : qui comporte une opération de translation par le vecteur $(x,y)$ et une symétrie centrale (inversion des coordonnées $i$ et $j$) : il s'agit du déplacement de l'image $g$ à la position $(x,y)$ ($g$ joue donc le rôle de fenêtre glissante),

[1] https://perso.esiee.fr/~perretb/I5FM/TAI/convolution/index.html -

- la multiplication *point à point* de $f$ par la translatée de $g$ : c'est l'opération de pondération,
- la sommation du tout : c'est l'opération de moyennage.

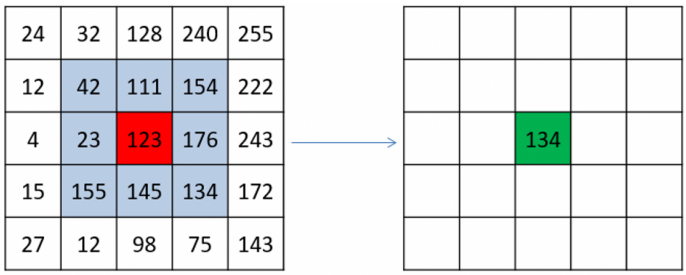



Le produit de convolution est une application bilinéaire, associative et commutative. C'est-à-dire que pour toutes images $f$, $g$, $h$ et pour tout scalaire $\lambda$, on a :

- $f*(g+\lambda h)=(f*g)+\lambda(f*h)$
- $(f*g)*h=f*(g*h)$
- $f*g=g*f$

Le choix du **noyau de convolution** va permettre d'obtenir différents effets (cf exemples page suivante).

L'idée du **filtre médian** reprend le principe de la fenêtre glissante mais ne s'intéresse pas à une combinaison linéaire des valeurs des pixels dans la fenêtre (contrairement au produit de convolution). Au lieu de cela, on va trier les pixels situés sous la fenêtre par ordre de niveau de gris croissant et prendre le pixel se trouvant au milieu : on parle d'élément ***médian***.Par exemple, si on considère la liste de valeur (1,3,6,9,223), cette liste est triée et comporte 5 éléments ; le médian de cette liste est le 3ème élément, c'est-à-dire le 6. Par rapport à la moyenne, le médian a l'avantage d'être robuste aux valeurs extrêmes qui ne sont pas forcément représentatives de la distribution des valeurs dans la liste. Dans le cas précédent, la moyenne vaut 48.4, elle est fortement influencée par la valeur 223 qui n'est pas vraiment représentative du reste des éléments de la liste. Alors que le médian à 6 semble effectivement mieux représenter les valeurs de la liste. Exemple ci-après de calcul avec un filtre médian de taille 3x3 :




$23 \leq 42 \leq 111 \leq 123 \leq 134 \leq 145 \leq 154 \leq 155 \leq 176$

L'opération de convolution est efficace pour débruiter une image et lisser les zones texturées. Malheureusement, elle a également une forte tendance à flouter les contours. Il existe une variation de l'opération de convolution appelée **filtre bilatéral** qui résout ce problème en introduisant une seconde pondération afin de ne pas donner trop de poids à un pixel dont la valeur est éloignée de la valeur du pixel courant. Cette méthode est décrite sur de nombreux sites Web, par exemple :
https://people.csail.mit.edu/sparis/bf_course/slides/03_definition_bf.pdf

### II.2. EXERCICES

On considère les images du fichier *imagesFiltres.zip*.

1. Créer un fichier ***filtrage_Q1_TP.cpp*** afin de pouvoir tester sur chacune des images les filtres moyenneur (***blur***), gaussien (***GaussianBlu***r) et médian (***medianBlur***). Consulter la documentation *opencv*[2] et un exemple d'utilisation dans le tutoriel [3]

**CR - Remplir le tableau ci-après en y incluant les images filtrées.**

**CR - Que peut-on déduire de ce tableau au sujet du comportement de ces filtres ?**

| Images | Filtre moyenneur 3x3 | Filtre moyenneur 5x5 | Filtre Gaussien 3x3 | Filtre Gaussien 5x5 | Filtre médian 3x3 | Filtre médian 5x5 |
|---|---|---|---|---|---|---|
| carre | | | | | | |
| casserole | | | | | | |
| NoisyMultBlobs | | | | | | |

[2] https://docs.opencv.org/3.4/d4/d86/group__imgproc__filter.html
[3] https://docs.opencv.org/3.4.2/dc/dd3/tutorial_gausian_median_blur_bilateral_filter.html

| | | | | | | |
|---|---|---|---|---|---|---|
| **NoisyAddBlobs** | | | | | | |

2. Compléter le fichier créé en 1. : ajouter un filtre bilatéral (***bilateralFilter***), tester plusieurs paramètres sur les fichiers *snack-orig.png* et *onion-ori.png* (consulter la documentation opencv[4] pour le choix des paramètres).

**CR –Préciser l'effet de ce filtre en illustrant avec les images filtrées et en donnant plusieurs exemples avec des paramètres différents dont vous commenterez les conséquences visuels.**

[4] https://docs.opencv.org/3.4/d4/d86/group__imgproc__filter.html#ga9d7064d478c95d60003cf839430737ed

## PARTIE II – COMPARAISON DE CONTOURS – DISTANCE D'EDITION - CODE DE FREEMAN (12 POINTS)

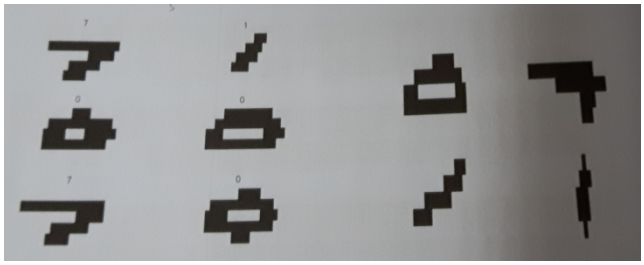
1. Programmer l'algorithme vu en cours d'extraction du contour de Freeman d'un objet dans une image binaire. Le tester sur les fichiers avec les extensions pgm proposés sur Arche à la séance 1.
   **Joindre le code à ce compte-rendu.**

2. Faire une recherche sur les algorithmes de ***Boyer et Moore (77)*** et de ***Wagner et Fischer(74)***.

   **CR - Donnez-en les principes et les algorithmes. Quelles en sont les limites selon vousdans la comparaison de contours codés avec le code de Freeman ?**

3. **Etude de l'algorithme de Wagner et Fischer**. Un code en Matlab de cet algorithme est disponible à l'adresse :
https://fr.mathworks.com/matlabcentral/fileexchange/17585-calculation-of-distance-between-strings.

On considère les images de la figure ci-contre dont vous trouverez les matrices ou les fichiers pgm dans le fichier *imagestests.zip*.



a. Déterminer les codes de Freeman des contours de chaque image (utiliser pour cela le programme créé en 1.). Remplir ensuite le tableau ci-dessous en indiquant **la distance d'édition**, résultat de l'algorithme de Wagner et Fischer, dans les cases vides entre les codes des contours concernés. Afin de diminuer l'impact des différentes longueurs des codes des contours, vous pourrez diviser chaque distance obtenue par la somme ou par le maximum des longueurs des deux codes comparés. Prendre un poids de 1 pour chaque opération substitution/insertion/suppression.

**CR - Mettre dans le compte-rendu 2 tableaux de résultats : sans et avec pondération.**
**Commenter les résultats obtenus.**

| | xt0_1 | xt7_1 | xt1_1 | xt1_2 |
|---|---|---|---|---|
| xa7_1 | | | | |
| xa1_1 | | | | |
| xa0_1 | | | | |
| xa0_2 | | | | |
| xa7_2 | | | | |
| xa0_3 | | | | |

b. Proposer une variante du calcul dans l'algorithme en mettant un poids différent selon les codes de Freeman au niveau des substitutions/insertions/suppression.

**CR - Commenter les résultats obtenus. Quelle solution vous semblerait mieux adaptée ?**